What Will Probably Be on the Exam – TWUN13

Based on the official HT2026 exam format from your teacher + the 3 old exams + your 9 lectures. E1 = WEB25-19-25, E2 = 2026-01-14 (newest), E3 = re-exam 2025-01-09.

1. The official exam format for HT2026 (from your teacher)

Your teacher published a **provisional exam format** on Canvas. This is the most important information you have – it replaces the pattern from the old exams. *(Lärarens officiella upplägg – viktigare än de gamla tentorna.)*

#	Topic	Points NOW	Old exams (E1/E2)	Change	Lectures
1	Web applications in general	2	3	↓	L07
2	HTML / CSS / JS	3	5	↓	L01, L02, L03
3	**Front end programming** (React)	**10**	5	**↑↑ doubled**	L04, L05, L06b
4	**Backend programming** (Node, Express)	**7**	5	↑	L07, L09
5	**Browser/server communication**	**9**	4	**↑↑ more than doubled**	L07, L09
6	Storing data in web applications	3	3	=	L09
7	Pentesting, cyber security	2	2	=	**not in your files**
8	Large scale applications & performance	3	2	↑	**not in your files**
9	State management	2	2	=	L04, L06a
10	**Code comprehension**	**9**	19–22	**↓↓ less than half**	L02, L03, L05
	Total	**50**	50		

Grade	NEW (HT2026)	Old exams
U (fail)	**< 30**	< 20
3	**30–39**	20–29
4	**40–45**	30–39
5	**46–50**	40–50

Trap: Passing is much harder now: you need 30 of 50 points (60%), not 20. You can no longer pass on the code questions alone – you need points from every topic. The ~12 "memorize" points (general 2, storing data 3, security 2, performance 3, state 2, HTML/CSS/JS part) are the easiest to secure.

Remember: The teacher is new. The old exams were written by someone else, so their exact pattern may not repeat. Use them for practice, but the **new lectures (Fall 2026)** – and especially the **quiz questions inside the slides** (section 5) – are now the best guide to how he will ask.

What the big topics probably contain

Topic (points)	Most likely content, based on the lectures
Front end 10	JSX → createElement, functional components, props (read-only, object), useState rules, useEffect + dependency array + cleanup, re-render rules, lists + keys, Fragments, Virtual DOM, declarative, React Router (Link, NavLink, useParams, useSearchParams, useNavigate, Outlet, loader/useLoaderData, action/Form, deep linking).
Browser/server communication 9	Client–server, HTTP stateless, request/response message structure (first lines = browser), **methods** (GET/POST/PUT/PATCH/DELETE), **status codes** (200, 201, 204, 400, 401, 403, 404, 500), headers (Content-Type, Accept, Authorization), URL parts + ports 80/443, JSON, API, **fetch** (res.ok trap, POST with body), **CORS** (+ credentials: 'include'), cookies sent automatically, Postman.
Backend 7	Node.js = runtime, npm / package.json / node_modules, require, Express: app.get / app.use / app.listen, **middleware + next()**, req.query / req.params / req.body, res.send / json / status / end, "Cannot GET", validation, Passport auth (LocalStrategy, serializeUser, req.user, isAuthenticated, logout), hashing + salt.
Code comprehension 9	Probably 2–3 questions (3–5 points each): a JS trap (hoisting, coercion, scope, event loop) and React state/effect behaviour (infinite loops, useEffect timing, "what does this render").

2. Points per topic on the OLD exams

For comparison only – the new weights are in section 1. *(Bara för jämförelse.)*

Topic	E1	E2	E3	Avg	Priority
React (state, useEffect, render, Context, Router)	22	23	11	**19**	★★★ #1
Node.js / Express / middleware	5	4	10	6	★★★ #2
HTTP, web basics, URL, status codes, fetch, CORS	6	6	7	6	★★★ #3
JS core (==, +, hoisting, scope, functions)	6	5	2	4	★★★ #4
HTML / CSS / DOM / events	2	3	6	4	★★
Async / event loop / Promises	2	1	3	2	★★
Databases (MongoDB, NoSQL, Mongoose)	3	2	3	3	★★
Cookies / sessions / auth	0	1	3	1–2	★★
Security (STRIDE, injection, HTTPS)	2	2	2	2	★★ (easy points)
Performance (CDN, cache, load balancing)	2	2	2	2	★★ (easy points)
TypeScript	0	0	1	–	★
Your own project (E3 only)	0	0	2	–	★

(E2 Q3 was an empty question, 1 point. E3 had 52 points with the optional project question.)

3. Questions that came back on several exams

These are the safest bets. The same idea was asked 2–5 times. *(Samma idé har kommit flera gånger – plugga dessa först.)*

Theme	Where	What to know
setState → infinite re-render	E1 Q33, E2 Q12, E2 Q33	setState in the body = "Too many re-renders" error. useEffect that sets the state it depends on = endless loop.
useEffect dependency array	E1 Q11, E1 Q35, E2 Q11, E2 Q13, E3 Q12	none = every render, [] = once, [x] = when x changes. Cleanup = returned function.
Hoisting with var	E1 Q32 (4p), E2 Q8	Declaration moves up, value doesn't → undefined.
Event loop order	E1 Q3, Q7, Q8, E2 Q17, E3 Q37	sync → Promise → setTimeout. var in loop → 3 3 3. async returns a Promise right away.

Theme	Where	What to know
Middleware	E1 Q16, Q17, Q18, E2 Q16, E3 Q15	app.use, (req,res,next), runs in order, next() = go on, can end the cycle.
Context	E1 Q12, Q31, E2 Q30, Q31	Solves prop drilling, Provider gives the value, all users re-render on change.
CORS purpose	E1 Q21 = E2 Q21 (identical!)	Lets the browser access APIs on other domains.
fetch + HTTP errors	E1 Q22, E2 Q22	fetch doesn't reject on 404/500 → check res.ok yourself.
Type coercion	E1 Q6, E2 Q32 (3p)	5 == "5" → true. "5" + 1 → "51".
"What does this render?"	E1 Q34, E2 Q34 (5p each)	Simple JSX: && / ternary / {name}. Say the HTML + one reason.
"Explain why…" React	E1 Q35, E2 Q35 (5–6p)	Timer + cleanup / state kept between renders + prev => prev + 1.
Node.js / Express definition	E1 Q14, E2 Q14, E3 Q14	Node = runtime for JS on server. Express = Node framework for web apps/APIs.
MongoDB	E1 Q23–25, E2 Q24–25, E3 Q23–24	NoSQL document DB, collection, _id, schema for query patterns, Mongoose model.
STRIDE	E1 Q27 (R), E2 Q26 (S), Q27 (I), E3 Q27 (D)	One letter per exam. **T (Tampering) and E (Elevation of privilege) have not been asked yet** → good guess for next time.
CDN / caching / load balancing	E1 Q28, E2 Q28–29, E3 Q31	Nearest server / store temp data / spread traffic.
CSS selectors	E1 Q5, E2 Q5	id > class > tag. `div p` vs `div>p` vs `div+p`.
HTTP status codes / methods	E1 Q19, E2 Q19, E3 Q18–20	PUT = replace all. 404 not found, 4xx client, 5xx server.

4. Prediction for code comprehension (now 9 points)

On E1 and E2 there were 4 code questions (19–22 points) with the **same pattern** every time. Now there are only **9 points**, so expect **2–3** of these types. Some React code will probably also move into the Front end part as multiple choice. *(Färre kodfrågor nu, men samma typer.)*

Question	Pattern on E1 / E2	Most likely next time
Q32 (3–4p) JS basics	E1: hoisting `var x` → undefined. E2: `"5" + 1` → "51".	A JS trap: hoisting, ==/===, string + number, var/let in loop, truthy/falsy, const object change, let → ReferenceError.
Q33 (5p) "What happens when this component renders?"	E1: useEffect sets count with [count] → endless loop. E2: setCount in body → Too many re-renders.	**Again an infinite loop variant**: useEffect with NO array that sets state, fetch in the body, or the correct version with [] (runs once).
Q34 (5p) "What does this render?"	E1: `show && <h1>`. E2: `<p>Hello {name}</p>`.	Simple JSX: ternary `? :`, `&&` with false or **0** (0 is rendered as "0"!), a list with map + key, props passed to a child.
Q35 (5–6p) "Explain why..."	E1: text disappears after 2 s (timer + cleanup). E2: count doesn't reset (state kept, prev => prev + 1).	useEffect cleanup, state kept between renders, **batching** (3× `setCount(count+1)` adds only 1, but 3× `setCount(p => p+1)` adds 3), props read-only.

Remember: How to write a T10 answer (max ~50 words): 1) the output in one line, 2) name the concept (hoisting / re-render / dependency array / cleanup / batching), 3) one sentence on the chain of events. Example: "Too many re-renders error. setCount runs in the body, so every render changes state, which starts a new render → infinite loop."

5. Questions from YOUR lecture slides (very likely to be reused)

Your teachers put quiz questions in the Fall 2026 slides. Teachers often reuse these on the exam. Know all the answers: *(Lärarna lägger ofta sina föreläsningsquiz på tentan.)*

Code (from L02 / L03)	Answer
`4 * "2" ** 3 + 10`	**42** (** first, "2" → 2)
`"100" == 100` / `"100" === 100`	**true** / **false**
`"Hello" && 0`	**0** (&& returns the first falsy value)
`let x; console.log(x);`	**undefined**
`const π = 3.14; π = 22/7;`	**TypeError**: Assignment to constant variable
`const squares = [1,4,9,16]; squares[4] = 25;`	**[1, 4, 9, 16, 25]** (const doesn't lock the content)
`user.hobbies` (property doesn't exist)	**undefined** (no error)
`user.age = undefined; console.log(user)`	**{name: "Lea Verou", age: undefined}** (not deleted)
`hobbies.user = "Lea"; hobbies.length` (array of 2)	**2** "Lea Verou" (arrays are objects)
`hobbies["1"] = "Dining"; hobbies[1]`	**Dining Dining** (index "1" = 1)
`if (true) { let x = 1; } console.log(x);`	**ReferenceError** (let is block-scoped)
`if(true){ let x=1; log(); } function log(){ console.log(x) }`	**ReferenceError** (lexical scope)
`Boolean("0"), Boolean([]), Boolean({})`	**true** (all truthy)
`setTimeout(hello, 0); console.log('I run...'); console.log('... very fast.')`	I run... → ... very fast. → When do I say hello?

Quiz from L05 (React)	Answer
What do props help us do?	Make components **reusable**.
How do you pass a prop?	`<MyHeader title="hi" />`
Can you put `blah={true}` on a `<div>`?	**No** – real DOM elements only have HTML attributes.
What data type is props?	An **object**.
Does changing a normal variable re-render?	**No** – only setState (or new props).

6. New in the Fall 2026 lectures – not asked yet, but could be

The slides were updated for this course round. These parts are new or bigger than before, so a new question here is possible:

- **React Router data mode**: `createBrowserRouter`, `loader` + `useLoaderData()`, `action` + `<Form>`, `<Outlet>`, `useParams()`, `useSearchParams()`, `useNavigate()`, `<NavLink>`.
- **Authentication**: authentication vs authorization, session flow, **hashing + salt**, Passport (`serializeUser`, `req.user`, `req.isAuthenticated()`), cookie attributes **HttpOnly / Secure / SameSite**, XSS vs CSRF, 401 on wrong login.
- **Databases**: **ACID**, **vertical vs horizontal scaling**, the 4 NoSQL types (key-value, column, graph, document), normalized vs denormalized, MongoDB `find()` vs SQL `SELECT`.
- **JS basics**: truthy/falsy, `&&` returns a value, `const` with objects, NodeList vs HTMLCollection, `textContent` vs `innerHTML` (security), `preventDefault`, `stopPropagation`, event delegation, `Promise.all` / `Promise.race`.
- **HTTP**: URL parts + default ports **80 / 443**, the request/response message structure, **201 Created, 204 No Content, 401 Unauthorized**, `req.query` vs `req.params` vs `req.body`.
- **React**: JSX → `React.createElement`, keys in lists, Fragments, Virtual DOM diffing, "lifting state up", the React 19 Context syntax `<MyContext value={...}>`.

7. Gaps: what the exams ask that your files don't cover

> **Trap: Lecture L08 is missing** (your files jump from L07 to L09). "Pentesting, cyber security" (2 points) and "Large scale applications & performance" (3 points) = **5 points on the new exam** but are in none of your 9 files. TypeScript and Mongoose/ORM were on E3. **Get L08 (and any security/performance slides) from the course page.** Until then, the last section of the summary PDF covers what the old exams asked.

8. Study plan by priority (new point weights)

Order	What	Points on new exam	Time
1	**React** (L04, L05, L06a, L06b): hooks rules, useEffect, re-render, props, keys, Router, Context	Front end 10 + State 2 + most of Code comp 9 ≈ **18–21**	35%
2	**Browser/server communication** (L07, L09): HTTP messages, methods, status codes, headers, URL, fetch, CORS, cookies	**9**	20%
3	**Backend** (L07, L09): Node, npm, Express, middleware, req/res, Passport, hashing	**7**	15%
4	**JS traps + event loop + HTML/CSS** (L01–L03)	HTML/CSS/JS 3 + part of Code comp	10%
5	**Memorize**: web general, storing data (MongoDB, SQL vs NoSQL, sessions), security (STRIDE, pentesting, SQLi, XSS, CSRF), performance (CDN, caching, load balancing, lazy loading, scaling)	2 + 3 + 2 + 3 = **10**	15%

Order	What	Points on new exam	Time
6	Redo the 3 old exams without looking at the answers	practice	5%

> **Remember:** Pass = **30 points**. Realistic plan: aim for **~70% in every topic** → about 35 points = grade 3 with a margin. Don't skip the 10 memorize points – they are the cheapest points on the exam.